

Universidad Politécnica Salesiana

Ingeniería en Ciencias de la Computación

Sistemas Distribuidos

Informe de Reflexión

Práctica CI/CD: Docker, Kubernetes y Blue-Green

Integrantes:

Gabriel Alexander Córdova Solórzano

Jordy Espinoza

Repositorio: <https://github.com/UPSGabriel/inventario-app-cicd-final>

Fecha: 26 de julio de 2026

1. Estrategia elegida: Blue-Green

Se eligió **Blue-Green** porque esta aplicación permite identificar la versión activa mediante el endpoint `/version`, que devuelve `version`, `color` y `hostname`. Kubernetes puede realizar el corte de tráfico usando únicamente dos `Deployment` y el selector de un `Service`.

BLUE conserva la versión `v1` mientras GREEN se valida con `v2`. Cuando GREEN está listo, el cambio de `slot=blue` a `slot=green` es inmediato. Si el smoke test falla, el rollback consiste en devolver el selector a BLUE, que sigue levantado.

Para una demostración de laboratorio esto es más determinista y fácil de evidenciar que Canary, donde sería necesario hacer muchas peticiones para observar una proporción estadística del reparto de tráfico.

Costo de la estrategia: ambas versiones consumen recursos al mismo tiempo. Además, la base de datos JSON local hace que cada pod tenga un catálogo independiente. En producción no usaríamos Blue-Green con este almacenamiento; moveríamos el catálogo a una base de datos externa antes de cortar tráfico.

El flujo del corte implementado es:

1. BLUE sirve `v1` a los usuarios.
2. GREEN `v2` arranca y supera el readiness probe.
3. El smoke test valida GREEN con `/health`, `/version` y `/api/products`.
4. El Service cambia su selector de `slot=blue` a `slot=green`.
5. Si hay problemas, el rollback consiste en cambiar el selector de vuelta a BLUE.

2. Observación sobre la persistencia de datos

`DB_PATH=/app/data/products.json` apunta a un volumen `emptyDir`. Ese volumen vive solo mientras vive el pod. Por eso, al crear un producto y luego eliminar ese pod, el `Deployment` crea otro pod con un `emptyDir` nuevo y la aplicación vuelve a cargar los tres productos semilla. El producto agregado desaparece.

Con dos réplicas también pueden observarse catálogos distintos según el pod que atienda cada petición, ya que cada pod tiene su propio archivo JSON independiente.

Es el comportamiento esperado de la arquitectura solicitada, no un error a corregir en esta práctica.

3. Métricas DORA propias

Los tiempos se calcularon con timestamps ISO-8601 conservados en Git y en el registro de despliegue. Se usó `git show -s --format="%H | %cI | %s"` para obtener la hora del commit y `Get-Date -Format o` para registrar el momento en que el cambio quedó corriendo en el clúster.

3.1. Lead time for changes

Cambio	Commit	Disponible en clúster	Lead time
Runtime distroless 285e565	23-jul 18:57:46	25-jul 21:34:57.195	50 h 37 min 11.196 s
Arranque lento a07b964	26-jul 00:44:14	26-jul 01:09:59.642	25 min 45.643 s

Cuadro 1: Lead time para los dos cambios desplegados durante la práctica.

Lead time promedio: $(50,6198h + 0,4293h)/2 = 25,5246h$

El primer lead time es alto porque el commit de **285e565** se hizo el miércoles y el despliegue en el clúster se completó el viernes, incluyendo tiempo dedicado a resolver el fallo de Trivy y configurar Minikube. El segundo cambio tuvo un lead time de 25 minutos porque el pipeline y el clúster ya estaban configurados.

3.2. Frecuencia de despliegue

Se realizaron **2 promociones exitosas** (cambios de imagen en el Deployment) en un período de **2 días calendario**, lo que da una frecuencia de **1 despliegue por día**.

Los cambios de selector Blue-Green no se cuentan como nuevos despliegues porque no cambian la imagen de los Deployments; solo redirigen tráfico entre versiones ya disponibles.

3.3. Change failure rate

De los 3 intentos de despliegue registrados durante la práctica, **1 requirió corrección** (el rollout que alcanzó su `progressDeadlineSeconds` por problemas de permisos del volumen):

$$Changefailure rate = \frac{1}{3} = 33,3\%$$

3.4. Reflexión sobre las métricas

Frente a la tabla de niveles vista en clase, la frecuencia de una vez por día y el lead time promedio cercano a un día muestran un flujo rápido para un laboratorio. El 33.3 % de fallos indica que la estabilidad todavía debe mejorar. La muestra es pequeña, por lo que estos valores describen esta práctica específica y no el rendimiento sostenido de un equipo.

4. Problemas reales encontrados y solución

1. **Vulnerabilidades críticas en la imagen inicial.** Trivy hizo fallar el run 29980646258 por vulnerabilidades CRITICAL en la imagen base. Se cambió el runtime final a `distroless` y el siguiente run (30054762604) terminó correctamente antes de publicar la imagen en GHCR.
2. **Permisos con usuario no root.** La imagen `distroless` usa UID/GID 65532 y no podía escribir el JSON en el volumen. Se agregaron `runAsUser`, `runAsGroup` y `fsGroup` con valor 65532, manteniendo `readOnlyRootFilesystem: true` y el volumen escribible solo en `/app/data`.
3. **Readiness durante el arranque lento.** `/health` devuelve HTTP 503 durante 12 segundos. El `readinessProbe` tolera ese periodo con `failureThreshold: 10` y `periodSeconds: 2`, mientras que el `livenessProbe` comienza después con `initialDelaySeconds: 20`. Aumentar réplicas no resolvería una sonda mal configurada: solo crearía más pods todavía no listos.
4. **Reproducción y cambio de tráfico.** Se corrigió el orden del README para crear el Secret antes del Deployment. Para el corte Blue-Green, `kubectl patch` con JSON era interpretado incorrectamente por PowerShell, así que se usó `kubectl set selector` que evita el problema de comillas.

5. Conclusión

El pipeline aplica *fail-fast*: las pruebas deben pasar, Trivy bloquea vulnerabilidades CRITICAL y solo entonces se publican las etiquetas SHA y `latest` en GHCR. Kubernetes aporta `RollingUpdate` para el despliegue base y Blue-Green para un corte y rollback rápidos.

Los tres componentes adicionales (Secret, Trivy y arranque lento) refuerzan las buenas prácticas de seguridad y disponibilidad. La principal deuda técnica es la persistencia local; una siguiente iteración debería externalizar la base de datos e integrar el smoke test como verificación automática posterior al despliegue.